

Universidad del Valle de Guatemala
 CC2008 - Introducción a la Programación Orientada a Objetos
 David Maximiliano Pablo Cuy

Análisis

1. Tabla de propiedades (atributos)

#	Clase	Atributo	Tipo de dato	Visibilidad	Propósito
1	Cliente	nombre	String	private	Almacena el nombre del propietario del vehículo
2	Vehiculo	placa	String	private	Almacena la placa del vehículo
3	OrdenServicio	numeroOrden	int	private	Identifica de forma única la orden
4	OrdenServicio	cliente	Cliente	private	Asocia la orden con su propietario
5	OrdenServicio	vehiculo	Vehiculo	private	Asocia la orden con el vehículo atendido
6	OrdenServicio	descripcionServicio	String	private	Describe el servicio a realizar
7	OrdenServicio	costoEstimado	double	private	Almacena el costo estimado del servicio
8	GestorOrdenes	ordenes	List<OrdenServicio>	private	Colección dinámica que almacena todas las órdenes activas
9	ControladorOrdenes	gestor	GestorOrdenes	private	Administra la colección de órdenes y la lógica de negocio
10	ControladorOrdenes	teclado	Scanner	private	Lee la entrada del usuario desde consola

2. Tabla de métodos

#	Clase	Método	Parámetros	Retorno	Visibilidad	Propósito
1	Cliente	Cliente()	String nombre	constructor	public	Crea el cliente con su nombre
2	Cliente	getNombre()	—	String	public	Retorna el nombre del cliente
3	Cliente	setNombre()	String nombre	void	public	Actualiza el nombre del cliente
4	Vehiculo	Vehiculo()	String placa	constructor	public	Crea el vehículo con su placa

#	Clase	Método	Parámetros	Retorno	Visibilidad	Propósito
5	Vehiculo	getPlaca()	—	String	public	Retorna la placa del vehículo
6	Vehiculo	setPlaca()	String placa	void	public	Actualiza la placa del vehículo
7	OrdenServicio	OrdenServicio()	int numeroOrden, Cliente cliente, Vehiculo vehiculo, String descripcionServicio, double costoEstimado	constructor	public	Crea la orden con todos sus datos
8	OrdenServicio	getNumeroOrden()	—	int	public	Retorna el número de orden
9	OrdenServicio	getCliente()	—	Cliente	public	Retorna el cliente asociado
10	OrdenServicio	getVehiculo()	—	Vehiculo	public	Retorna el vehículo asociado
11	OrdenServicio	getDescripcionServicio()	—	String	public	Retorna la descripción del servicio
12	OrdenServicio	setDescripcionServicio()	String descripcion	void	public	Modifica la descripción del servicio
13	OrdenServicio	getCostoEstimado()	—	double	public	Retorna el costo estimado
14	OrdenServicio	setCostoEstimado()	double costo	void	public	Modifica el costo estimado
15	OrdenServicio	mostrarInformacion()	—	void	public	Imprime todos los datos de la orden
16	GestorOrdenes	GestorOrdenes()	—	constructor	public	Inicializa la lista vacía con new ArrayList<>()
17	GestorOrdenes	registrarOrden()	int numeroOrden, Cliente cliente, Vehiculo vehiculo, String descripcion, double costo	void (throws DatosInvalidosException, OrdenDuplicadaException)	public	Valida los datos y agrega una nueva orden a la colección
18	GestorOrdenes	consultarOrdenes()	—	void	public	Muestra todas las órdenes registradas
19	GestorOrdenes	buscarOrden()	int numeroOrden	OrdenServicio (throws OrdenNoEncontradaException)	public	Busca y retorna una orden por su número
20	GestorOrdenes	modificarOrden()	int numeroOrden, String nuevaDescripcion, double nuevoCosto	void (throws OrdenNoEncontradaException)	public	Modifica descripción y costo de una orden existente
21	GestorOrdenes	cancelarOrden()	int numeroOrden	void (throws OrdenNoEncontradaException)	public	Elimina una orden de la colección

#	Clase	Método	Parámetros	Retorno	Visibilidad	Propósito
				contradaException)		
22	GestorOrdenes	consultarPorPlaca()	String placa	List<OrdenServicio>	public	Retorna todas las órdenes asociadas a una placa
23	GestorOrdenes	calcularValorTotal()	—	double	public	Suma el costo estimado de todas las órdenes activas
24	GestorOrdenes	calcularCostoPromedio()	—	double	public	Calcula el promedio de costo de las órdenes activas
25	GestorOrdenes	ordenMayorCosto()	—	OrdenServicio	public	Retorna la orden con el costo estimado más alto
26	GestorOrdenes	cantidadOrdenes()	—	int	public	Retorna cuántas órdenes hay registradas
27	ControladorOrdenes	ControladorOrdenes()	—	constructor	public	Inicializa el gestor y el Scanner
28	ControladorOrdenes	iniciar()	—	void	public	Muestra el menú en bucle hasta que el usuario elija salir
29	ControladorOrdenes	mostrarMenu()	—	void	private	Imprime las 10 opciones del menú
30	ControladorOrdenes	registrarOrden()	—	void	private	Pide los datos por teclado y llama a gestor.registrarOrden() con try-catch-finally
31	ControladorOrdenes	consultarOrdenes()	—	void	private	Llama a gestor.consultarOrdenes()
32	ControladorOrdenes	buscarOrden()	—	void	private	Pide el número de orden y maneja OrdenNoEncontradaException
33	ControladorOrdenes	modificarOrden()	—	void	private	Pide los nuevos datos y llama a gestor.modificarOrden()
34	ControladorOrdenes	cancelarOrden()	—	void	private	Pide el número de orden y llama a gestor.cancelarOrden() con try-catch-finally
35	ControladorOrdenes	consultarPorPlaca()	—	void	private	Pide la placa y llama a gestor.consultarPorPlaca()
36	ControladorOrdenes	reporteDeCostos()	—	void	private	Llama a gestor.calcularValorTotal() y gestor.calcularCostoPromedio()
37	ControladorOrdenes	ordenMayorCosto()	—	void	private	Llama a gestor.ordenMayorCosto()

#	Clase	Método	Parámetros	Retorno	Visibilidad	Propósito
38	ControladorOrdenes	cantidadOrdenes()	—	void	private	Llama a gestor.cantidadOrdenes()
39	OrdenDuplicadaException	OrdenDuplicadaException()	String mensaje	constructor	public	Crea la excepción con un mensaje descriptivo
40	OrdenNoEncontradaException	OrdenNoEncontradaException()	String mensaje	constructor	public	Crea la excepción con un mensaje descriptivo
41	DatosInvalidosException	DatosInvalidosException()	String mensaje	constructor	public	Crea la excepción con un mensaje descriptivo
42	Main	Main()	String[] args	void	Public static	Punto de entrada

2. Responda las siguientes preguntas:

3. ¿Qué tipo deben tener las propiedades y métodos de cada clase?

Los datos de texto se manejarán como String, por ejemplo, el nombre, la placa y la descripción. Los números enteros, como el número de orden, serán de tipo int, mientras que los montos, como el costo estimado, serán de tipo double. Para la colección dinámica se utilizará List<OrdenServicio>.

4. ¿Qué información deberá almacenarse dentro de la colección dinámica?

La colección almacenará objetos completos de tipo OrdenServicio. Cada objeto tendrá asociado un Cliente y un Vehículo, además de la descripción del servicio y el costo estimado.

5. ¿Dónde utilizará List y dónde realizará la creación mediante ArrayList?

El atributo de GestorOrdenes se declarará utilizando el tipo general List<OrdenServicio>, pero se creará como un ArrayList en el constructor mediante new ArrayList<>(). De esta manera, se sigue la buena práctica de programar utilizando la interfaz en lugar de depender directamente de una implementación.

6. ¿Cuáles deben ser los modificadores de visibilidad de los miembros de cada clase?

Todos los atributos serán private para mantener el encapsulamiento. Los constructores y los métodos públicos de acceso o de negocio serán public. Los ControladorOrdenes, los métodos internos utilizados solo para manejar el menú serán private, ya que no necesitan ser accesibles desde otras clases.

7. ¿Qué parámetros serán requeridos por los métodos de sus clases?

El método registrarOrden recibirá todos los datos necesarios para crear una orden: número, cliente, vehículo, descripción y costo. Los métodos buscarOrden, modificarOrden y cancelarOrden recibirán el número de la orden. Además, modificarOrden recibirá la nueva descripción y el nuevo costo. Por último, consultarPorPlaca recibirá la placa del vehículo que se desea consultar.

8. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores iniciales les asignará?

Todos los objetos se inicializarán mediante su constructor, de manera que ninguno quede con atributos incompletos. En el caso de `GestorOrdenes`, la lista comenzará vacía utilizando `new ArrayList<>()`.

9. ¿Cómo identificará una orden específica dentro de la colección?

Cada orden se identificará por su `numeroOrden`, que será un valor único de tipo `int`. Para comparar este valor se utilizará `==`, ya que se trata de un tipo de dato primitivo.

10. ¿Cómo agregará y eliminará órdenes de la colección dinámica?

Para agregar una orden se utilizará `ordenes.add(orden)`. Para eliminarla, primero se recorrerá la lista con un `for` para encontrar la orden utilizando su número. Una vez encontrada, se eliminará por `ordenes.remove(ordenEncontrada)`.

11. ¿Cómo realizará la búsqueda de todas las órdenes asociadas a una misma placa?

Se recorrerá la lista utilizando un `for-each` y se comparará la placa de cada vehículo con la placa buscada mediante `orden.getVehiculo().getPlaca().equals(placaBuscada)`. Las órdenes que coincidan se irán guardando en una lista temporal para mostrarlas posteriormente.

12. ¿Cómo recorrerá la colección para calcular el valor total y el costo promedio de las órdenes?

Se utilizará un `for-each` para recorrer todas las órdenes y sumar el `costoEstimado` de cada una. El promedio se calculará dividiendo esa suma entre el tamaño de la lista, es decir, entre `ordenes.size()`.

13. ¿Cómo determinará cuál es la orden con el costo estimado más alto?

Se recorrerá la lista con un `for-each` y se irá guardando en una variable la orden que tenga el costo más alto encontrado hasta ese momento. Cada nueva orden se comparará con la que actualmente tenga el mayor costo.

14. ¿Qué situaciones dentro del programa pueden producir una excepción?

Algunas situaciones que pueden generar una excepción son intentar registrar un número de orden que ya existe, dejar vacío el nombre, la placa o la descripción, ingresar un costo menor o igual a 0, buscar, modificar o cancelar una orden que no existe, o ingresar texto cuando el programa espera un número.

15. ¿Qué operaciones serán protegidas utilizando `try-catch`?

La operación de registrar una orden estará protegida mediante `try-catch` para controlar las validaciones y la lectura del costo. También se utilizará en buscar, modificar y cancelar una orden, principalmente porque existe la posibilidad de que el número de orden ingresado no exista.

16. ¿Qué acción realizará mediante el bloque `finally` y por qué debe ejecutarse independientemente del resultado de la operación?

Al finalizar cada operación del menú, como buscar o cancelar una orden, el bloque `finally` mostrará un mensaje indicando que la operación terminó y permitirá regresar al menú principal. Esto debe ejecutarse independientemente de si ocurrió una excepción o no, para que el programa pueda continuar funcionando y el usuario pueda seguir utilizándolo sin que se detenga inesperadamente.